

Research on AGV Path Planning Based on Deep Reinforcement Learning in RMFS Scenarios

Bao-Qi Sun^{1*} and Ke-Ming Wang¹

¹School of Computer Science and Artificial Intelligence, Southwest Jiaotong University, Chengdu, 610036, Sichuan, China.

*Corresponding author(s). E-mail(s): sunbaoqi@email.com;
Contributing authors: wangkeming@email.com;

Abstract

With the large-scale development of intelligent warehousing, efficient path planning for Automated Guided Vehicles (AGVs) in dynamic Robotic Mobile Fulfillment Systems (RMFS) has become a critical challenge. Traditional heuristic algorithms struggle to adapt to dynamic environments, while standard Deep Reinforcement Learning (DRL) often faces issues of sparse rewards and convergence difficulties under complex constraints. To this end, this paper proposes a single-AGV path planning algorithm that fuses heuristic guidance with DRL, termed DRS-DQN (D* Lite Reward Shaping-DQN). This method innovatively utilizes the dynamic distance field maintained by the D* Lite algorithm as a potential function to construct a back-tracking reward shaping mechanism, providing dense reward gradients while ensuring the invariance of the optimal policy. To address the issue of outdated experience in dynamic environments, a delayed calculation mechanism and Prioritized Experience Replay (PER) are introduced. Experiments conducted in five RMFS simulation environments of varying scales (54 to 156 storage stations) and layouts (rectangular and fishbone) demonstrate that DRS-DQN stably converges and completes tasks with a 100% success rate. In contrast, standard DQN fails to converge in the same scenarios. Furthermore, the experiments confirm that the fishbone layout significantly outperforms the traditional rectangular layout in terms of reinforcement learning training efficiency.

Keywords: Automated Guided Vehicle, Path Planning, Deep Reinforcement Learning, Reward Shaping, Dynamic Environment, Robotic Mobile Fulfillment System (RMFS)

1 Introduction

Currently, the domestic logistics industry in China is experiencing rapid development. As a pivotal link, warehouse logistics is undergoing a dual transformation characterized by massive scale expansion and profound technological innovation. In terms of scale, the total area of general warehousing in China climbed to approximately 1 billion square meters in 2025, representing a year-on-year increase of 0.30%. With the number of warehouse parks reaching 40,750, the industry demonstrates a vigorous and sustained growth trend.

In the context of the rapid advancement of intelligent manufacturing and smart logistics, intelligent transformation has emerged as the core trajectory for modern warehousing. Numerous enterprises are actively deploying advanced technical equipment to enhance operational efficiency and management precision. Automated Guided Vehicles (AGVs), as quintessential representatives of this intelligent shift, are widely utilized in smart warehouses due to their superior automation and intelligent capabilities. AGV systems not only substitute for high-intensity, high-risk manual labor but also enable material tracking and optimized scheduling through integration with high-level control systems.

However, as the scale of AGV applications expands, the operating environment becomes increasingly complex. In dynamic industrial scenarios, AGVs must navigate among fixed shelves and equipment while avoiding randomly appearing pedestrians, forklifts, and other moving AGVs. The efficiency and adaptability of their path planning directly determine the overall system performance. Therefore, finding a collision-free, short-distance, and low-energy path for AGVs in complex dynamic environments has become a core research topic in the field of mobile robotics.

Traditional AGV path planning relies on classic algorithms such as A*, Dijkstra, and RRT. However, as warehouse logistics upgrades toward "intelligence, dynamism, and scale," the limitations of these traditional methods become apparent: they exhibit poor adaptability to dynamic environments, depend heavily on static pre-set maps, and suffer from high decision latency when environments change. Furthermore, their multi-AGV coordination capabilities are insufficient; in multi-AGV scenarios, graph search algorithms face exponential scheduling complexity, while the randomness of sampling algorithms often leads to congestion and poor synergy.

Recently, Deep Reinforcement Learning (DRL) has provided a promising direction for AGV path planning by enabling agents to learn autonomously through interaction with the environment. DRL does not require a pre-defined environment model and can adapt to dynamic changes by fitting complex features via neural networks. For example, DQN addresses high-dimensional state space issues through experience replay, while PPO enhances training stability through policy clipping.

To address the challenges of poor adaptability in intelligent warehouse AGV path planning, this study proposes the ****DRS-DQN (D* Lite Reward Shaping-DQN)**** algorithm—a single-AGV path planning framework that integrates D* Lite heuristic guidance with DRL. The core contributions are as follows:

1. We introduce back-tracking reward shaping based on potential functions, which effectively solves the problems of sparse rewards and slow convergence in DRL training without altering the optimal policy.
2. We propose the DRS-DQN algorithm, which incorporates D* Lite heuristic information into the DQN framework. We design a four-matrix state space, a three-category action reward structure, and a four-layer neural network to achieve efficient path planning.
3. We introduce a dynamic task mechanism and multi-layout training strategies to enhance generalization and robustness. Experiments in various RMFS simulation environments demonstrate stable convergence and a 100% task success rate.

The remainder of this paper is organized as follows: Section 2 reviews the related work; Section 3 details the methodology and framework of the DRS-DQN algorithm; Section 4 presents the experimental setup and analyzes the results; and Section 5 concludes the paper and discusses future research directions.

2 Related Work

2.1 Research Background

Path planning is a cornerstone technology in the evolution of robotic systems. In warehouse environments, path planning algorithms must supply automated guided vehicles (AGVs) with collision-free trajectories while satisfying optimization criteria such as energy consumption, transport time, and communication latency [?]. The objective of AGV path planning research is to navigate an AGV from its origin to its destination in as few steps as possible, avoiding all obstacles along the way, thereby improving the overall performance of the warehouse system.

2.1.1 Traditional Path Planning Algorithms

A substantial body of research has addressed this domain, yielding a variety of traditional algorithms that broadly fall into two categories: global path planning and local path planning [?].

Global path planning algorithms assume that the robot possesses complete knowledge of the obstacle layout in its surroundings. These methods first construct an abstract map model of the global environment, then perform a global search over this model [?] to derive an optimal or near-optimal path that safely guides the robot to its target. Representative global path planning algorithms include graph search methods, sampling-based methods, and bio-inspired intelligent algorithms. Graph search methods, such as Dijkstra’s algorithm [?] and A* [?], compute a feasible path from start to goal based on known map information; however, the state-space search paradigm underlying these methods incurs poor computational efficiency in complex scenes due to the large volume of data and computation time required. Sampling-based planning algorithms, with the Rapidly-exploring Random Tree (RRT) algorithm as a prominent example, are particularly well-suited to single-AGV path planning in static environments, yet they also contend with low search efficiency and susceptibility to

local optima. Bio-inspired intelligent algorithms, including genetic algorithms, particle swarm optimization, and ant colony optimization, solve path planning problems by simulating natural phenomena such as biological evolution, foraging behavior, and nest construction. These methods search for optimal solutions by mimicking individual evolution and collective swarm behavior, which makes them suitable for multi-robot systems and yields relatively high solution efficiency. In practice, however, they are prone to converging to local optima and exhibit slow convergence speed.

Local path planning algorithms are designed for scenarios in which the robot can perceive only partial environmental information through its sensors. The robot must autonomously plan a collision-free optimal path in real time based on the locally sensed data. Owing to their reliance on local environmental features, these methods may produce only locally optimal paths rather than globally optimal ones, and in some cases may fail to reach the destination altogether [?]. Widely adopted local path planning algorithms include the artificial potential field (APF) method [?] and the dynamic window approach (DWA) [?]. The APF method models the robot as moving within a virtual force field, where obstacles exert repulsive forces and the target exerts an attractive force; the resultant of these forces guides the robot through the environment. While computationally simple and capable of real-time obstacle avoidance, the APF method is susceptible to local minima and may exhibit oscillatory behavior in narrow passages [?]. The DWA is a velocity-based local planning method that imposes velocity constraints to satisfy kinematic and obstacle-avoidance requirements, searching for the optimal control velocity in the velocity space to achieve swift and safe navigation. Its success rate, however, depends heavily on global parameters, and it is prone to failure in unfamiliar environments.

Traditional algorithms have been widely applied to warehouse robot path planning. Wang et al. [?] proposed a complex diagonal distance metric to address the shortcomings of conventional distance functions in A*, reducing both the number of searched nodes and the planned path length. Li et al. [?] introduced PQ-RRT*, a novel algorithm that achieves both asymptotic optimality and rapid convergence, overcoming the slow convergence of the original RRT. Liu et al. [?] developed an improved ant colony algorithm incorporating an adaptive pheromone evaporation coefficient strategy and a load-balancing strategy, which enhanced the algorithm’s search efficiency. Wang et al. [?] constructed virtual target points on top of the original APF method and combined them with a fuzzy control approach to design a finite state machine, improving the robot’s adaptability in complex environments. Xiao et al. [?] proposed the IACO-DWA algorithm for robot path planning in large-scale complex environments, refining the evaluation function of the traditional DWA to further improve path safety and smoothness.

In summary, while traditional path planning algorithms have achieved considerable research progress, they remain subject to fundamental limitations. They typically rely on pre-constructed environmental map models, which compromises their effectiveness in dynamic and complex settings. Furthermore, most of these methods are designed to plan paths from a fixed start point to a fixed goal point, rendering them incapable of accommodating real-time changes in the target destination. This means the robot

is effectively restricted to a locally optimal solution, making these algorithms ill-suited to the dynamically changing environments and tasks characteristic of intelligent warehouses.

2.1.2 Reinforcement Learning-Based Path Planning Algorithms

Reinforcement learning (RL) is an emerging direction in artificial intelligence research. Through interaction with the environment, an RL agent receives feedback signals in the form of rewards or returns and makes a sequence of decisions aimed at maximizing cumulative reward. Owing to their independence from pre-specified environmental map models and prior knowledge, RL algorithms have increasingly become a research focus for path planning in various complex environments.

Notable progress has been made in applying RL to path planning. Soong et al. [?] proposed a partially guided Q-learning algorithm that employs the flower pollination algorithm (FPA) to improve the initialization process, addressing the problem of slow convergence to the optimal solution. Zhao et al. [?] introduced an experience-memory Q-learning algorithm that continuously updates the shortest distance from the current state node to the starting point; by using an EM table to record distance information and a Q-table to assist in guiding experience transfer and experience reuse strategy updates, the algorithm improves the robot's learning efficiency. Xu et al. [?] proposed an improved Q-learning algorithm that incorporates a gravitational potential field during Q-value initialization and adds directional factors to the state set; experimental results demonstrate that the improved algorithm requires less planning time and produces smoother paths. Wang [?] integrated the heuristic function of the A* algorithm into the action selection strategy of conventional Q-learning, thereby accelerating convergence speed.

To address the pervasive issue of sparse rewards in RL, Andrychowicz et al. introduced the Hindsight Experience Replay (HER) mechanism. The core idea of HER is reward relabeling: states that the agent actually reaches, even if they are not the intended goal, are retroactively designated as virtual goals, thereby extracting useful learning signals from failed experiences. Although HER performs well with static goals, extending its relabeling paradigm to topologically dynamic environments where obstacles change over time remains a challenging open problem.

2.2 Summary and Problem Analysis

A synthesis of the above research on traditional path planning algorithms and reinforcement learning in the domain of AGV scheduling reveals that, while existing methods have achieved meaningful results in specific scenarios, they still face intractable theoretical and engineering bottlenecks when confronted with the highly dynamic, high-density characteristics of Robotic Mobile Fulfillment Systems (RMFS). The limitations of current research concentrate on the following three fronts.

First, there is a fundamental conflict between the optimality guarantees of traditional heuristic algorithms and the overhead of dynamic replanning. Traditional global path planning algorithms such as A* and Dijkstra can reliably compute collision-free globally optimal paths in known static environments. RMFS scenarios, however, are

characterized by extreme dynamic complexity: AGVs inevitably encounter randomly moving pedestrians, other operating AGVs, or temporarily scattered materials during execution. Under such highly dynamic conditions, once the environmental topology changes due to emergent obstacles, traditional graph search algorithms must perform high-frequency replanning for the affected nodes or even the entire global map. As the system scale expands and AGV density increases, the computational overhead of such replanning grows exponentially, engendering severe communication and decision-making delays that substantially constrain the overall throughput of the warehouse system.

Second, standard RL suffers from acute sparse reward and convergence difficulties under complex constraints. Deep reinforcement learning, with its paradigm of offline training and online decision-making, offers a promising alternative for dynamic obstacle avoidance. Algorithms such as standard DQN can produce action commands at the millisecond level, yet they exhibit fatal shortcomings when deployed in practical RMFS settings. RMFS imposes stringent physical operating rules, for instance that fully loaded AGVs are prohibited from traversing storage stations and non-target picking stations. These rules cause the state-action space to be saturated with illegal actions. During the prolonged early exploration phase, it is exceedingly difficult for the agent to trigger the terminal reward of task completion through random exploration alone. This severe sparse reward problem, compounded by an extremely high illegal action rate, leads to the accumulation of massive amounts of ineffective noisy data in the experience replay buffer. The neural network is unable to extract effective policy gradients from such data, ultimately resulting in extremely slow convergence or outright failure of the model.

Third, existing reward shaping mechanisms exhibit clear limitations in dynamic topological scenarios. To mitigate the sparse reward problem in RL, the research community widely adopts reward shaping mechanisms to provide dense learning signals. Mainstream studies, however, predominantly rely on static potential functions, such as those derived from the artificial potential field method or the heuristic distance of a static A* planner. In RMFS environments, the random movement of dynamic obstacles can sever the original optimal topological corridor at any moment. If static potential-based reward shaping is retained under these conditions, the static potential field not only fails to provide correct obstacle-avoidance guidance when the optimal path is temporarily blocked, but also persistently outputs erroneous gradient signals that direct the agent to traverse obstacles. Such misleading rewards can cause the AGV to oscillate locally near obstacle boundaries, enter deadlock, or even increase the risk of collision.

Taken together, these observations indicate that the field of AGV path planning for complex dynamic RMFS scenarios still lacks an effective integrative framework. Such a framework must simultaneously preserve the global optimal guidance capability of classical heuristic algorithms in complex topological structures and fully harness the millisecond-level online responsiveness of deep reinforcement learning when confronting emergent dynamic obstacles.

To address this gap, this paper proposes a single-AGV path planning algorithm termed DRS-DQN (D* Lite Reward Shaping-DQN), which is guided by the dynamic

potential of the D* Lite distance field. Chapter 3 elaborates on how this algorithm achieves obstacle avoidance and stable convergence in dynamic warehouse environments through backtracking reward shaping and a deferred computation mechanism, while ensuring policy optimality.

3 Method

This chapter describes how the proposed DRS-DQN algorithm collaborates with the Robotic Mobile Fulfillment System (RMFS) model to achieve path planning. Section 3.1 introduces the DQN algorithm architecture tailored to the RMFS path planning problem. Section 3.2 elaborates on the reward shaping mechanism. Section 3.3 details the construction of reward shaping in dynamic scenarios. Section 3.4 presents the overall framework and training techniques of DRS-DQN.

3.1 RMFS Environment Modeling

In a Robotic Mobile Fulfillment System (RMFS), environment modeling serves as the foundation for AGV path planning, directly determining whether the robot can accurately acquire information about its own position, task objectives, and the surrounding environment. RMFS scenarios are characterized by regularly arranged shelving and highly structured operational workflows. To precisely describe the topology of the RMFS environment space while accounting for the convenience of map construction and the frequency of AGV path planning, this study adopts a grid map for formal modeling of the RMFS simulation environment. This approach partitions the space into uniform cells, each storing a value that represents regions such as shelves, picking stations, and traversable aisles, thereby achieving discrete representation of the continuous workspace. The resulting RMFS environment model not only captures the real warehouse layout but also provides a consistent and trainable state representation for deep reinforcement learning algorithms such as DQN.

3.2 DQN Algorithm for AGV Path Planning

In a DRL-based path planning model, three core elements constituting a Markov Decision Process (MDP) must be designed to suit the RMFS context: state, action, and reward. In addition, the neural network architecture must be appropriately specified.

3.2.1 State Space

In reinforcement learning, the state or observation encapsulates the internal and external information and logic of the environment. Within the RMFS, an operating AGV can access a wealth of global information, including the physical layout of the scene (i.e., the specific positions of picking stations, storage stations, and tracks), its own current position, the target position, and its operating status.

To enable the deep neural network to effectively process and learn these environmental features, this study constructs four matrices of exactly the same dimensions as the RMFS model, each serving as one component of the system state. Each entry

in these matrices takes only two possible values: 0 for false and 1 for true. The specific definitions of the four matrices are as follows.

The Valid Location Matrix (VLM) represents all positions that the AGV can legally enter at the current moment. If the AGV can move into a given cell, the corresponding entry is set to 1; otherwise, it is set to 0. The determination rules are as follows. Track regions always permit AGV passage, so track cells are invariably set to 1. The value for a storage station cell depends on the AGV’s loading state: when the AGV is empty, it is permitted to pass beneath the storage station, and the entry is set to 1; when the AGV is carrying a shelf (fully loaded), passage through the storage station is prohibited, and the entry is set to 0. A picking station, being a scarce resource, has its cell set to 1 only when it is the AGV’s current task target; in all other cases, it is set to 0.

The Forbidden Location Matrix (FLM) takes values that are the exact complement of the VLM, serving as supplementary information to indicate which positions are accessible or inaccessible. Specifically, a position with value 1 in the VLM has value 0 in the FLM, and vice versa. Introducing this pair of complementary matrices strengthens the feature representation of the current environment and provides more informative inputs to the neural network.

The Current Location Matrix (CLM) is constructed by first initializing an all-zero matrix of the same size as the RMFS model, then setting only the cell corresponding to the AGV’s current position to 1, leaving all other entries as 0.

The Target Location Matrix (TLM) is constructed in the same manner as the CLM: starting from an all-zero matrix, only the cell representing the AGV’s current task target position is set to 1.

Through this four-matrix state space design, the system transforms the discrete warehouse operating environment with dynamic constraints into tensor data of dimensions $4 \times W \times H$ (where W and H denote the map width and height), which can be directly used as input features for a convolutional neural network.

3.2.2 Action Space

In the RMFS, shelves are arranged in a regular grid pattern and AGVs move primarily along straight aisles. Considering both algorithmic complexity and policy stability, this paper restricts AGV actions to four discrete directions —up, down, left, and right—and maintains a constant linear velocity. Since neural networks can only output numerical results, each action is assigned a numeric encoding. During system operation, whenever the AGV enters a new cell, it sends observation information to the Python program; the DQN algorithm returns an action to the AGV, which then checks whether the action is legal. A legal action is executed and the system continues running; an illegal action causes the system to terminate. The action space is $A = \{\text{Up, Down, Left, Right}\}$. The rules for determining action legality are introduced in the following section, together with the reward structure.

3.2.3 Reward Structure

The reward is defined based on the outcome of the action taken by the AGV as prescribed by the DQN algorithm, and it serves to evaluate the value of that action.

Actions fall into three categories: illegal actions, correct actions, and normal actions, each associated with a distinct reward value.

Illegal actions refer to actions that cannot be executed; performing such an action causes the RMFS model to halt. An action is classified as illegal in the following three cases: (1) the action drives the AGV out of the system boundary; (2) the action causes a loaded AGV to collide with a storage station; (3) the action leads the AGV into a picking station that is not its final destination. The reward for an illegal action is -100 .

A correct action is one after which the AGV reaches its final destination, for instance arriving at a designated storage station to retrieve or deposit a shelf, or reaching a designated picking station. The reward for a correct action is $+1$.

A normal action is one that does not cause the RMFS model to terminate and after which the AGV has not yet reached its final destination. The reward for a normal action is 0 . Normal actions and correct actions are collectively referred to as legal actions.

$$r(s, a) = \begin{cases} -100, & a \text{ is illegal} \\ +1, & s' \text{ is target} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

3.2.4 Neural Network Architecture

The neural network (Deep Q Network) used in this study adopts a five-layer architecture, with three convolutional layers followed by two linear layers. Convolutional layers are well-suited for extracting features from image pixels and matrices and for identifying latent regularities therein. Each convolutional layer contains multiple neurons with learnable weights and biases; the number of neurons must be adjusted according to the dimensions of the input matrices, with larger matrices requiring more neurons and smaller matrices fewer. The linear layers, also known as fully connected layers, transform the dimensionality of the preceding layer’s output, fit a large volume of input information according to linear relationships, and produce the final output. The number of neurons in a linear layer depends on the number of neurons in the preceding layer.

3.3 Reward Shaping

As the term suggests, reward shaping aims to accelerate and stabilize learning by modifying the reward directly obtained from the environment. It works by adjusting the reward of the Markov Decision Process (MDP) to guide the learning algorithm toward desired behavior. The general form of shaped reward is:

$$R' = R + F \quad (2)$$

where $R : S \times A \times S \rightarrow \mathbb{R}$ is the expected reward function of the original MDP, and $F : S \times A \times S \rightarrow \mathbb{R}$ is the expected reward shaping function. Summing the two yields the modified expected reward function $R' : S \times A \times S \rightarrow \mathbb{R}$. Under this

formulation, the learning process operates on the modified reward R' rather than the original reward R , and seeks a policy that maximizes the discounted sum of R' .

A substantial body of successful cases has demonstrated that reward shaping is an effective tool for steering an agent toward desired behavior (Brys et al., 2014; Devlin et al., 2011; Dorigo and Colombetti, 1994; Efthymiadis and Kudenko, 2013; Mataric, 1994). However, if applied inappropriately, reward shaping can alter the optimal policy of the MDP and induce unintended behavior. A canonical example was provided by Randløv and Alstrøm (1998): a bicycle was tasked with riding from point A to point B, and the agent received a reward whenever it moved toward B, with all other rewards set to zero. The agent learned a strategy of circling around point A rather than advancing toward the goal. This example raises a natural question: to what extent can the reward be adjusted without changing the optimal policy of the original problem? This condition is known as policy invariance.

Ng et al. (1999) established how to adjust the reward while guaranteeing policy invariance. They introduced a potential function $\Phi_S : S \rightarrow \mathbb{R}$ that maps each state to a scalar value. Desirable states (e.g., those reached under the target policy) are assigned higher function values, and less desirable states are assigned lower values. The potential function draws inspiration from the physical concept of potential energy, i.e., the energy of an object in a given state relative to a reference state. Using this potential function, Ng et al. (1999) defined the potential-based reward shaping function $F : S \times A \times S \rightarrow \mathbb{R}$ as:

$$F(s_t, a_t, s_{t+1}) = \gamma \Phi_S(s_{t+1}) - \Phi_S(s_t) \quad (3)$$

where γ is the discount factor and Φ_S denotes the potential function over states. Ng et al. (1999) proved that any policy that is optimal in the modified MDP $\langle S, A, T, \gamma, R + F \rangle$ is also optimal in the original MDP $\langle S, A, T, \gamma, R \rangle$, where S is the state space, A is the action space, T is the state transition probability, R is the original reward function, and F is the reward shaping function. Notably, as long as F takes the specific potential-based form given above, policy invariance holds regardless of the choice of Φ_S .

Wiewiora, Cottrell, and Charles (2003) extended the work of Ng et al. (1999) by proposing a potential function that depends on both state and action, denoted $\Phi_{S,A} : S \times A \rightarrow \mathbb{R}$. This new potential function provides more specific information to guide the agent toward desired behavior, since it becomes possible to assign values to state-action pairs rather than only to states. Building on $\Phi_{S,A}$, Wiewiora et al. (2003) proposed two potential-based reward shaping functions, termed look-ahead reward shaping and look-back reward shaping. The look-ahead variant resembles the method of Ng et al. (1999) in that it uses the potential of the next state-action pair (s_{t+1}, a_{t+1}) ; the look-back variant instead uses the potential of the previous state-action pair (s_{t-1}, a_{t-1}) in the reward shaping function $F : S \times A \times S \times A \rightarrow \mathbb{R}$. This study adopts the latter, defined as:

$$F(s_t, a_t, s_{t-1}, a_{t-1}) = \Phi_{S,A}(s_t, a_t) - \gamma^{-1} \Phi_{S,A}(s_{t-1}, a_{t-1}) \quad (4)$$

In general, it is difficult to directly substitute Equation (4) into Equation (2), because the former depends on the policy being followed. In other words, with this

form of reward shaping function, one cannot simply construct a new MDP with modified reward $R + F$ and theoretically prove policy invariance. A notable exception occurs when the policy being followed is stationary (see Wiewiora et al. (2003) for the relevant theoretical proof). The reason is that, in expectation, the potential of the previous state-action pair is computable when the policy is fixed. Hence, under a stationary policy π , one can define $R' = R + F^\pi$ (where F^π denotes the dependence on the stationary policy being followed) and policy invariance can be shown to hold.

In this study, the deep Q-network is updated in every training iteration, which means the policy being followed is not stationary; consequently, policy invariance cannot be theoretically established. Nevertheless, subsequent empirical results demonstrate that the shaped DQN algorithm can sometimes outperform the teacher policy, suggesting that policy invariance may in fact hold, although no formal proof is currently available.

This study adopts the look-back potential-based reward shaping method of Wiewiora et al. (2003) and defines the potential function $\Phi_{S,A}$ according to a teacher policy. A known heuristic algorithm is suggested to serve as the teacher policy to guide the learning of the DQN algorithm. Let $a^\dagger(s)$ denote the action recommended by the teacher policy in state s . The potential function $\Phi_{S,A}$ is defined as:

$$\Phi_{S,A}(s, a) = -k |a^\dagger(s) - a| \quad (5)$$

where $k \in \mathbb{R}^+$ is an arbitrarily chosen parameter that amplifies the guiding effect of the teacher policy. Thus, the greater the discrepancy between the action a taken in state s and the action $a^\dagger(s)$ recommended by the teacher, the lower the potential function value. Substituting this potential function into the look-back reward shaping function of Wiewiora et al. (2003) (Equation (4)) indirectly encourages the agent to take actions consistent with the teacher policy.

At present, there is no evidence that the resulting policy converges to the optimal policy when a neural network is used to approximate the value function. Accordingly, prior work on potential-based reward shaping has concentrated primarily on tabular reinforcement learning. This study applies potential-based reward shaping to deep reinforcement learning; although the relevant theoretical guarantees cannot be derived, the empirical results indicate that policy invariance still holds in the deep RL setting.

3.4 Dynamic Potential-Based Reward Shaping

3.4.1 Theoretical Foundation: From Static to Dynamic

Conventional potential-based reward shaping (PBRs) assumes that the potential function $\Phi(s)$ is static. In dynamic RMFS scenarios, however, the real-time movement of obstacles alters the optimal path topology. If a static A* heuristic distance is retained under these conditions, the guidance signal becomes ineffective or even misleading.

To address this problem, this study draws on the dynamic potential-based reward shaping (Dynamic PBRs) theory proposed by Devlin and Kudenko (2012). They proved that the conventional PBRs formulation can be relaxed: as long as the shaped reward F follows a specific differential form in which the potential function $\Phi(s, t)$

varies dynamically with time t , the optimal policy remains unchanged. The general form of Dynamic PBRS is defined as:

$$F(s, t, s', t') = \gamma \Phi(s', t') - \Phi(s, t) \quad (6)$$

where s and s' are states, t and t' are the corresponding time steps, and γ is the discount factor. This theory provides the theoretical guarantee for incorporating D* Lite dynamic planning into deep reinforcement learning.

3.4.2 Potential Function Based on D* Lite Distance Field

Unlike A*, D* Lite maintains a $g(s)$ value and an $rhs(s)$ value for each node to identify the impact of environmental changes on the path. The $g(s)$ value represents the path cost from the current node s to the goal, and changes only when the algorithm explicitly updates the node. The $rhs(s)$ value is the current best estimate computed from the $g(s')$ values of all neighboring nodes s' plus the transition cost $c(s, s')$ between them. It reflects the actual potential distance from node s to the goal after accounting for the current environment, for instance whether new obstacles have blocked the path.

The core advantage of D* Lite lies in its incremental replanning capability. When edge costs in the environment change, for example upon detecting a dynamic obstacle, D* Lite does not restart the search from scratch. Instead, it leverages the priority queue preserved from the previous search and updates only those inconsistent nodes for which $g(s) \neq rhs(s)$. This allows the algorithm to repair only the affected local region rather than recomputing the global path.

Furthermore, D* Lite typically performs a backward search from the goal toward the starting point. This means the algorithm maintains a goal-centered distance field. Regardless of the AGV's current position, D* Lite can provide a real-time estimate of the true cost from that position to the goal.

Unlike conventional approaches that use teacher actions for guidance, this study utilizes the distance field maintained by the D* Lite algorithm as the source of potential. D* Lite is an incremental heuristic search algorithm specifically designed for rapid replanning by reusing prior search results when the environment undergoes partial changes.

We define the dynamic potential function $\Phi(s, t)$ at time t as the negative correlation of the path cost computed by D* Lite:

$$\Phi(s, t) = -\omega(\eta) \cdot g_{D^*}(s, t) \quad (7)$$

where $g_{D^*}(s, t)$ denotes the actual path cost (i.e., the rhs value) from the current state s to the goal as maintained by the D* Lite algorithm at time t . When dynamic obstacles appear, D* Lite updates the g values of only the affected nodes, thereby correcting the distance field in real time at low computational cost. The term $\omega(\eta)$ is a potential weight coefficient that varies dynamically with the training progress, and η denotes the current training episode.

Substituting this potential function into the shaping formula (Equation (6)) yields the final shaped reward F_t received by the agent:

$$F_t = \omega(\eta) \cdot [g_{D^*}(s, t) - \gamma \cdot g_{D^*}(s', t')] \quad (8)$$

The physical interpretation of this formulation is that the agent receives a positive potential-difference reward when it moves toward a region with a smaller g value, i.e., closer to the goal, thereby forming a dense gradient-based guidance reward.

3.4.3 Deferred Computation of Dynamic Potential and Prioritized Experience Replay

In a dynamic environment where conditions vary over time, the dynamic potential function at time t is $\Phi(s, t)$. The conventional practice is to store the shaped reward $R' = R + F$ directly into the experience replay buffer. Owing to the offline nature of DQN, however, when an old sample is later retrieved for training, its stored potential-based guidance may no longer be consistent with the current environment state, for instance because dynamic obstacles have since moved. In such cases, the potential-based guidance computed at storage time diverges from the current environment: rather than providing correct heuristic guidance, it delivers an erroneous reward gradient that misleads the neural network update.

To resolve the problem of outdated experience in dynamic environments, we consider the reward relabeling method from Hindsight Experience Replay (HER). The replay buffer stores only the state s , action a , next state s' , and the original environmental reward r . When the network performs a gradient descent step, the current D* Lite map is queried to compute $\Phi(s, t)$ and $\Phi(s', t')$, from which F is calculated and added to the target Q-value. This deferred computation mechanism ensures that, regardless of how long a sample has been stored, it always reflects the most recent and accurate obstacle-avoidance topology guidance at the moment it is used for updating the network parameters.

When computing the loss function, each sample is weighted by its importance sampling weight. In standard DQN, samples are drawn uniformly at random, and the distribution of samples observed by the network matches the true distribution of the environment. With Prioritized Experience Replay (PER), however, samples with high priority (i.e., large TD-error) are artificially drawn more frequently. For a sample whose selection probability $P(i)$ is low, a proportionally larger weight is assigned so that the loss function can be properly evaluated:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \quad (9)$$

3.5 Overall Architecture of the DRS-DQN Algorithm

Fig. 1 Flowchart of the DRS-DQN algorithm.

A neural network can learn to fit high-dimensional inputs through iterative training, but several hyperparameters influence the training process and may even determine whether the network converges. Among these, the learning rate and the exploration rate are the most critical; they exert a particularly strong influence on the training process and are discussed in detail below.

Learning rate. The learning rate, together with the loss value, determines the magnitude of changes to the neural network parameters. Most reinforcement learning algorithms, including DQN, employ gradient descent to optimize network weights, relying on two quantities: the learning rate and the gradient. The iterative weight update rule is given by Equation (10), where θ_j denotes the weight and $\frac{\Delta J(\theta)}{\Delta \theta_j}$ denotes the gradient. A learning rate that is too small slows the learning process, whereas a learning rate that is too large may accelerate training but increases the risk of divergence or failure to converge. The gradient is computed from the cost function and the loss value. The loss is defined in Equation (11), where $r_t + \gamma \cdot \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$ represents the network’s prediction of the current state value based on the actual reward r_t and the expected future reward $\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$, while $Q(s_t, a_t)$ represents the network’s full prediction of the current state value. The difference between these two terms captures the discrepancy between the reward-based state value and the full predicted state value, which constitutes the loss.

$$\theta_j = \theta_j - lr \cdot \frac{\Delta J(\theta)}{\Delta \theta_j} \quad (10)$$

$$loss = r_t + \gamma \cdot \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \quad (11)$$

Exploration rate. The exploration rate governs the probability of using the neural network for decision-making and determines when to stop relying on the heuristic algorithm for exploration; once exploration ceases, all decisions are made exclusively by the neural network. A higher exploration rate corresponds to a higher probability of using the neural network for decisions, while $1 - \text{exploration rate}$ represents the probability of using the D* Lite algorithm for decisions. For instance, when the exploration rate is 0.8, the neural network makes decisions with 80% probability and the D* Lite algorithm with 20% probability. The training objective is for the neural network to converge to a policy that makes correct decisions based on input observations. Accordingly, the D* Lite algorithm is invoked more frequently in early training, and its usage is gradually reduced as training progresses. It is critical that reliance on the D* Lite algorithm be terminated sufficiently early so that the neural network is eventually responsible for all decisions, which is necessary for convergence. The exploration rate schedule is as follows, where the horizontal axis denotes the training episode (the total number of episodes varies with the scale of the RMFS). During the first 30% of episodes, the exploration rate is fixed at 0.8. From 30% to 80% of episodes, the exploration rate linearly increases from 0.8 to 1.0. Beyond 80% of episodes, the exploration rate is fixed at 1.0, at which point the D* Lite algorithm no longer participates in the decision-making process and all actions are determined by the neural network.

Termination condition. The termination condition defines when the training process concludes. In this study, training does not terminate after a fixed number of episodes; instead, it terminates when the neural network achieves a prescribed number of consecutive correct decisions. Once this threshold is reached, the training process ends. The threshold is adjusted according to the scale and complexity of the RMFS to ensure that the neural network can fully learn and achieve convergence.

4 实验

4.1 实验设置

4.2 实验结果与分析

5 结论

结论内容。